When Evolutionary Routing Fails: Action Complementarity and Utility Prediction in Structured Decision Models

Anonymous ACL submission

Abstract

Structured decision models such as Jev turn a language-model call into a typed probabilistic judgment: a binary NOUL decision, a candidate CHOICE, or a SCORE. This interface makes inference-time routing attractive: ask a cheap NOUL question first and buy a second, differently framed CHOICE view only when the first answer looks uncertain. We test that intuition with frozen open models. Using Laya 0.4.16 and Vela 2.0 0.3B on CodeXGLUE defect detection, FinDVer claim verification and ProcessBench reasoning-error localisation, we compare a genetic algorithm, an evolution strategy and NSGA-II against fixed policies and budget-matched random search, and we measure the two quantities routing requires: per-example action complementarity and the predictability of the second action's incremental utility. Gross repair opportunities exist—a per-row oracle gains up to 23.3 accuracy points over the better fixed action—but net repair is sparse and pool-dependent: only one of four confirm cells has a positive accuracy interval, and its macro-F1 interval still spans zero. Probability features rank repairable rows above chance in some pools (out-of-fold AUROC up to 0.92), yet frozen thresholds do not gain on held-out data. The genetic algorithm achieves no significant win over random search in 60 paired comparisons, and its five significant training-pool wins over fixed switching are not reproduced on dev and confirm. Search plateaus at generation zero in 15 of 21 runs. Temperature scaling reduces calibration error but cannot create routing signal, because monotone recalibration leaves the band policy class unchanged. Routing value requires complementary actions and a deployable estimate of incremental action value; calibrated confidence supplies neither by itself.

1 Introduction

A claims analyst asks a model whether a sentence is supported by a financial report; a security engineer asks whether a C function contains a vulnerability; a mathematician asks where the first invalid step occurs. In each case the downstream system wants a decision it can branch on, not a paragraph a human must parse. System One models answer that demand directly. Given a state and a typed question, Jev returns a binary NOUL, a candidate CHOICE, or an ordered SCORE with probability information, and the calling program can act on the result without a second interpretation step (TypeSafe AI, 2026; Deußer et al., 2026). Laya and Vela expose similar interfaces in open weights (Laya Project, 2026; vllm-sr, 2026). Uncertainty becomes a first-class value that software can read.

That value suggests an obvious control policy. Ask the cheap binary question first, inspect the returned probability, and spend a second call on a differently framed CHOICE question only when the first answer looks shaky. The policy is easy to implement, needs no labels at deployment, and matches how practitioners already use confidence thresholds. It also rests on an assumption that is rarely stated: that the probability attached to the current decision tells us something about the value of a different decision. A model can be unsure and still be right, or confident and about to switch from a correct NOUL answer to an incorrect CHOICE one. What matters for routing is not $P(\text{current answer is correct})$ but the incremental utility of the alternative action, $U(\text{CHOICE} \mid x) - U(\text{NOUL} \mid x)$.

The two quantities are easy to conflate because both are probabilities over the same label set, but they are learned from different supervision. RLCD, the recipe behind Jev and the one Laya documents, optimises the honesty of the reported distribution with strictly proper scoring rules (TypeSafe AI, 2026; Laya Project, 2026; Gneiting and Raftery, 2007). It teaches the model to say 0.7 when it is right about seventy percent of the time. Nothing in that objective asks the model to predict whether

a CHOICE question would have produced a better answer, so a well-calibrated head can coexist with a router that has no useful signal. Cascades inherit the problem: Jitkrittum et al. (2023) show that confidence-based deferral is optimal only under restrictive assumptions about how downstream errors relate to upstream confidence, and that specialised downstream errors, label noise or distribution shift can break the rule.

We measure the distinction instead of assuming it. Two structured decision models, Laya 0.4.16 and Vela 2.0 0.3B, are frozen, and inference-time routing is studied on three tasks with different decision interfaces: CodeXGLUE defect detection (Lu et al., 2021), FinDVer financial claim verification (Zhao et al., 2024) and ProcessBench reasoning-error localisation (Zheng et al., 2025). On the binary tasks the action set is {NOUL, CHOICE}: the policy always pays for NOUL and may pay again for CHOICE. On ProcessBench it decides whether to answer or abstain from the top-1 step, using candidate probability and the top-1/top-2 margin. Band policies are searched with a genetic algorithm (GA), an evolution strategy (ES) and NSGA-II, each compared with fixed policies and with random search under the same number of unique candidate evaluations. All outputs are generated once on CPU and replayed from an immutable cache, so every comparison sees exactly the same rows.

The design separates three questions. RQ1 asks whether the action set contains net complementarity: does the second action fix more rows than it breaks? RQ2 asks whether the incremental utility of the second action is predictable from signals available before it is executed. RQ3 asks whether evolutionary search converts whatever signal exists into a policy that beats fixed and random baselines on held-out data. A negative answer to RQ1 makes RQ3 uninteresting; a positive RQ1 with a negative RQ2 points to a representation or supervision problem rather than an optimiser problem. Two boundary checks, cross-task transfer and a confidence-stratified fault map, test whether any conclusion is an artefact of one pool or one configuration.

The results are consistently negative, and the way they fail is informative. Gross complementarity is real: on CodeXGLUE with Laya, an oracle that picks the better action per row gains 23.3 accuracy points over the better fixed action, and CHOICE repairs 14 NOUL errors. It also breaks 15 correct answers, so net repair on the training pool is -1. Held-out pools do not stabilise the picture. One confirm cell, FinDVer with Laya, shows a positive net-repair accuracy interval, but its macro-F1 interval spans zero and the better fixed action reverses between dev and confirm. Utility prediction repeats the pattern: out-of-fold AUROC reaches 0.92 for FinDVer with Vela, yet the threshold chosen on dev produces a negative net utility on confirm, and the best dev policy for CodeXGLUE with Laya degenerates to switching on every row. Search does not rescue the signal: GA has no significant win over random search in any of 60 paired macro-F1 comparisons, its five significant wins over fixed switching all occur on the FinDVer/Vela training pool and are not reproduced on dev or confirm, and NSGA-II deployment is significantly worse than the fixed baseline in six cells. Calibration is poor—confirm ECE ranges from 0.09 to 0.29—but temperature scaling leaves the routing outcome unchanged or worse, because a monotone recalibration cannot create signal the band policy class could not already express.

The paper contributes a diagnostic decomposition of routing value into action complementarity, incremental-utility predictability and search efficacy, measured on frozen structured decision models rather than inferred from end-task accuracy. It reports a pre-registered negative result for evolutionary routing with paired group-bootstrap intervals across five seeds, three pools and four task–model cells, including the training-pool-only nature of the apparent wins. It shows that calibration and action-value prediction dissociate, and locates the failures in the action set and the supervision target rather than in optimiser capacity.

2 Background and Related Work

Structured decision models replace free-form generation with a typed question and a typed answer. Jev, the commercial System One model from TypeSafe, accepts a state and a set of questions and returns NOUL (a binary judgment), CHOICE (a selection among named candidates) or SCORE (an ordered rating) with associated probabilities (TypeSafe AI, 2026). Deußer et al. (2026) benchmark Jev on 37 datasets, and Guo et al. (2026) apply it to alignment failure detection; both treat the model as a general decision interface rather than a text generator. TypeSafe describes the training objective as Reinforcement Learning for Calibrated Decisions (RLCD), which optimises the reported distribution with strictly proper scoring rules so that probabili-

ties carry information beyond the argmax; the full recipe is not public. Gao and Wang (2026) provide an independent open implementation. The name RLCD also appears in Yang et al. (2023) for Reinforcement Learning from Contrast Distillation; the two are unrelated, and we use RLCD only in the calibrated-decision sense.

The two models we run differ in backbone and supervision while sharing the interface. Laya 0.4.16 is a ModernBERT-large encoder with decision-specific heads, about 421M parameters, and supports NOUL, CHOICE and SCORE in a non-autoregressive pass; it is the open model closest to the RLCD description (Laya Project, 2026; Laya Studio, 2026). Vela 2.0 0.3B is a 22-layer ModernBERT encoder with an 8,192-token window; it exposes the same three actions plus SPAN and SET, and was initialised from the Decision-1.0-Kai CHOICE path with task-specific losses and routing data (vllm-sr, 2026; Warner et al., 2024). Vela's card does not call the recipe RLCD, so we treat Laya as an open RLCD model and Vela as an interface-matched comparison with a different training route (Appendix A).

Calibration is the natural first lens on these probabilities. Neural networks are often overconfident, and temperature scaling or other post-hoc maps can reduce expected calibration error without changing the argmax (Guo et al., 2017). Language models can also partially recognise their own errors, which makes confidence a plausible routing feature (Kadavath et al., 2022), and proper scoring rules formalise why a reported probability should be rewarded for honesty rather than sharpness alone (Gneiting and Raftery, 2007). All of this work concerns the probability of the answer the model has already produced, not whether that probability orders the counterfactual value of a different question type. We measure ECE and Brier for current-answer correctness, then measure the same probability's ability to rank rows that the alternative action would repair.

Cascades and routers allocate computation across models or actions. The classical cascade rejects easy examples early and sends hard ones to a stronger classifier (Viola and Jones, 2001); modern systems learn when to call a larger language model (Chen et al., 2024; Aggarwal et al., 2024; Ong et al., 2024), how many candidates to sample (Ding et al., 2025), or how to combine routing and cascading in one objective (Dekoninck et al., 2025). The theoretical caution comes from Jitkrittum et al. (2023): confidence-based deferral is optimal only when downstream errors are sufficiently tied to upstream confidence, and it can fail under specialised downstream errors, label noise or distribution shift. Damani et al. (2025) argue that computation should follow expected benefit rather than surface difficulty. Our experiments instantiate this warning in a structured decision model: the second action is a different question to the same frozen network, not a larger model, and its value depends on an error structure that the first probability does not encode. For ProcessBench, selective prediction supplies the abstention framing: Chow (1970) derives the optimal reject rule under known distributions, and Geifman and El-Yaniv (2017), Kamath et al. (2020) and Xin et al. (2021) study selective classification, question answering and the separation of coverage from conditional accuracy.

Evolutionary methods are a reasonable choice when the policy is a small non-differentiable object and the objective is a cached replay. Genetic algorithms, evolution strategies and NSGA-II cover single-objective and Pareto formulations (Deb et al., 2002; Hansen and Ostermeier, 2001; Stanley and Miikkulainen, 2002), and regularised evolution has been used for architecture search (Real et al., 2019; Elsken et al., 2019). In NLP, EvoPrompt and Promptbreeder evolve prompts or mutation instructions with language models in the loop (Guo et al., 2024; Fernando et al., 2024). Those systems search over text that changes what the model sees; we hold the model and the question template fixed and search over a two-parameter gate that decides whether a second typed question is asked. Prompt search can improve a single call, whereas our search can only exploit complementarity that already exists between two fixed actions. A flat landscape is therefore a finding about the action set, not merely about optimiser strength.

Our protocol follows the variance-aware reporting practices of Henderson et al. (2018), Bouthillier et al. (2021) and Dodge et al. (2019): we fix the action set, templates and budget before looking at confirm, report every seed-level interval rather than only the best cell, and label the locked test as not run rather than substituting a confirm number for it.

3 Method and Decision Rules

We study a frozen model M and a fixed action set $\mathcal{A}$. An input x is rendered by a task adapter into a

state string and a typed question; the model returns a schema-validated decision with a probability. The policy reads only the first call and decides whether to issue a second call. Model weights, question templates and candidate label sets are never modified during search. This isolates inference-time control from fine-tuning the backbone on the routing objective.

For the binary tasks, CodeXGLUE and FinDVer, the first action is NOUL: the model answers a yes/no question and returns $p(x) = P(\text{true} \mid x)$. The second action is CHOICE: the model selects between two named labels. In CodeXGLUE the questions are whether a function contains a vulnerability and whether it should be labelled `secure` or `vulnerable`; in FinDVer they are whether a claim is supported and whether it should be labelled `supported` or `not supported`. The cascade always pays for NOUL first and pays for CHOICE only when the gate opens. A policy is a genome $g_{\text{cas}} = (\ell, h, u)$ with $\ell, h \in [0, 1]$ and $u \in \{0, 1\}$:

$$\pi_g(x) = \begin{cases} \text{CHOICE}(x), & u = 1 \wedge \ell \le p(x) \le h, \\ \text{NOUL}(x), & \text{otherwise.} \end{cases} \tag{1}$$

The gate uses the positive-class probability rather than symmetric confidence, because a NOUL question has a designated true label. Genome repair clips ℓ and h to $[0, 1]$, swaps them when $\ell > h$, and maps u to $\{0, 1\}$. Two fixed policies are special cases: $u = 0$ is NOUL-only, and $u = 1, \ell = 0, h = 1$ is fixed switch-only, which still pays for the first NOUL call and therefore costs two calls per row.

ProcessBench has a different action space. The model returns a ranked list of candidate steps, and the policy decides whether to answer with the top-1 step or to abstain. A genome $g_{\text{abs}} = (t, v, m)$ with $t, m \in [0, 1]$ and $v \in \{0, 1\}$ uses the top-1 probability $s_1(x)$ and the top-1/top-2 margin $\Delta(x)$:

$$\text{abstain}(x) = \begin{cases} \text{true}, & s_1(x) < t, \\ \text{true}, & v = 1 \wedge \Delta(x) < m, \\ \text{false}, & \text{otherwise.} \end{cases} \tag{2}$$

When the required confidence fields are missing, the evaluator fails closed: the row is forced to answer and is counted as incorrect rather than silently dropped. We report coverage (the fraction of rows answered) and selective accuracy (accuracy on answered rows). The two genomes never share an action set or a cost model, so we never pool their accuracy numbers.

Fitness combines task quality with the cost of the second call:

$$J(g) = Q(g) - \lambda\, C(g), \tag{3}$$

where Q is macro-F1 on the binary tasks and step accuracy on ProcessBench, $C(g)$ is the mean number of model calls per row minus one, and $\lambda = 0.05$ is fixed before search. Macro-F1 is primary because both binary datasets are class-imbalanced; accuracy is reported alongside it. The first NOUL call is charged to every policy, including NOUL-only, so the cost term isolates the incremental call rather than the total inference bill. Model outputs are generated once and replayed from an immutable JSON cache, which makes candidate evaluation deterministic and cheap. We measure end-to-end latency for the native actions but do not claim it for every searched policy.

The GA uses tournament selection, uniform crossover, Gaussian mutation and bit flips on u. The ES is a $(\mu + \lambda)$ strategy that mutates the two endpoints and fixes $u = 1$. NSGA-II maintains a non-dominated front, optimising accuracy and negative call cost on the binary tasks and selective accuracy and coverage on ProcessBench; because its quality objective is accuracy rather than macro-F1, we keep its intervals separate. The main budget is 24 unique genome evaluations with seeds 17, 23 and 41; a five-seed extension adds 59 and 73. A probe raises the CodeXGLUE/Vela budget to 48. Random search uses the same budget and cache.

One property of the policy class shapes the calibration experiment. The gate depends on $p(x)$ only through the interval $\ell \le p \le h$. Any strictly monotone transformation of p—temperature scaling, for example—maps the interval to another interval and induces the same set of switched rows up to a reparameterisation of the endpoints. A monotone recalibration therefore cannot change the best achievable band policy on a fixed pool. It can move rows between bands, but it cannot create a distinction between repairable and harmful rows that the raw ordering did not already contain. Calibration and routing are therefore evaluated separately.

4 Experimental Design

The experiments use three tasks whose labels and group structure are fixed before any policy is searched. CodeXGLUE defect detection provides 21,854 training, 2,732 dev and 2,732 test functions; the controller pools are drawn only from the official

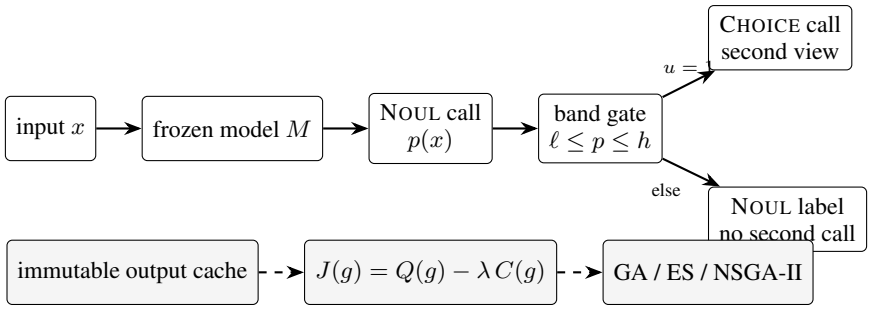


Figure 1: The routing setting. A frozen model answers a first NOUL question; a two-parameter gate decides whether the same model is asked a second CHOICE question. Search operates entirely on cached outputs: a candidate genome is scored by replaying the cache, and the optimiser only sees the scalar fitness J. The dashed path is offline; the solid path is the deployed policy.

training split, and functions are grouped by an exact hash of their text so that identical functions cannot cross the internal `search_train`, `selection_dev` and `confirm` pools (Lu et al., 2021; Zhou et al., 2019). Each internal pool contains 60 functions. This controls duplicate-function leakage but is not a project-level holdout. FinDVer provides financial claims with `supported`/`not supported` labels; its pools are grouped by report, and contain 60, 59 and 44 claims (Zhao et al., 2024). The official FinDVer test labels are not used for controller search. ProcessBench contains 3,400 annotated records over 2,842 mathematical problems; its pools are grouped by problem and contain 60 records each (Zheng et al., 2025; Cobbe et al., 2021; Lightman et al., 2024; Uesato et al., 2022). The task is to identify the earliest erroneous step or to certify that the chain is correct.

All inference runs on one Windows machine with CPU-only PyTorch in FP32, two threads and one model resident at a time. Laya is pinned to source revision `b9fe351` and checkpoint revision `7b928d8`; Vela is pinned to revision `d6f03aa`. Peak resident memory stays below 2.2 GB. The native single-action baselines on `search_dev` are reported in Appendix A: CodeXGLUE accuracy is 0.450/0.650 for Laya (NOUL/CHOICE) and 0.633/0.667 for Vela; FinDVer is 0.712/0.644 for Laya and 0.458/0.458 for Vela; ProcessBench CHOICE accuracy is 0.10 for both models. The near-identical per-call latency of the two representations (about 2.2–2.8 s) is why call count, rather than a model-specific latency model, is the cost term in Equation 3. The native numbers already contain a warning: the better representation depends on the task and the model, and on FinDVer the two Vela actions are indistinguishable at the aggregate level.

E1 measures action complementarity without training a router. For each task–model cell we take the better fixed action on `search_train` and freeze that direction; we then count rows where the alternative action repairs a base error, rows where it breaks a correct base answer, and their difference. The oracle headroom is the accuracy of a per-row oracle minus the accuracy of the better fixed action. Because the direction is fixed on train, the held-out net-repair numbers are not selected on the evaluation pool. We report net repair in percentage points with a group-level percentile bootstrap (5,000 resamples), and pair it with the macro-F1 difference between fixed CHOICE and fixed NOUL on the same rows. ProcessBench has only one evaluated representation, so its headroom is zero by construction and it is excluded from the binary complementarity table.

E2 asks whether the incremental utility of the second action is predictable before that action is executed. The prediction target is whether switching would repair the current NOUL error. The features are deployable at the moment of decision: p, $|p - 0.5|$ and the binary entropy $-p \log p - (1-p)\log(1-p)$; no CHOICE output, gold label or downstream feature is used. We compare logistic regression, a shallow decision tree, a single confidence threshold and a never-switch majority baseline. Training uses group-aware three-fold out-of-fold predictions; the predictor and threshold are then chosen together on `selection_dev` by net utility, frozen, and evaluated once on `confirm`. Net utility is (repairs − regressions) divided by the number of rows; the cost-adjusted variant subtracts $\lambda = 0.05$ per extra call. A champion that switches every row is retained, because it reveals whether

the predictor adds anything beyond fixed switching.

E3 compares optimisers under a matched evaluation budget. GA, ES and NSGA-II are run on the cached pools with 24 unique genome evaluations and seeds 17, 23 and 41; the five-seed extension adds 59 and 73. Random search receives the same budget. The primary deployment comparison is pre-registered: the GA genome selected on `search_train` is frozen and compared with fixed switch-only on `confirm` using paired group-bootstrap macro-F1 intervals. All other cell-level comparisons are exploratory and are reported with effect estimates and intervals rather than as significance claims. E4 examines the search trajectory itself: best-so-far fitness per generation, the number of unique genomes actually evaluated, the fraction of single-gene mutations that leave fitness unchanged, the rank correlation between train and dev fitness across the archive, and the effect of raising the budget cap from 24 to 48 on CodeXGLUE with Vela. These diagnostics use existing logs and caches and add no model calls.

E5 checks two boundaries. The transfer experiment freezes each source-task genome and applies it to every target task and pool without re-searching; diagonal cells are same-task deployment checks, off-diagonal cells are true transfer. The fault map partitions rows by task, model and a confidence band derived from NOUL probability or candidate margin, and reports the error severity of each cell on discover, dev and confirm. A cell is eligible when it contains at least eight rows on the discover pool, and it is confirmed when the dev or confirm severity is within 0.15 of the discover severity.

Two statistical details matter for reading the tables. All binary comparisons use paired group bootstrap with 5,000 resamples and a fixed random state (Efron and Tibshirani, 1993); the resampling unit is the group (function hash or report), not the row, so dependence within a report is preserved. We report McNemar exact tests for repair/harmful counts (McNemar, 1947) and Wilson intervals for within-cell severity (Wilson, 1927), but we do not call a result significant on the basis of a single seed or a single pool. The locked test has not been run. The confirm pools are held out from search and threshold selection but have been inspected during analysis; a locked evaluation would require freezing templates, policy class and statistics first.

5 Results and Discussion

The first question is whether the action set offers anything to route. Table 1 and Figure 2 (Appendix A) answer it at the level of individual rows. Gross complementarity is not scarce. On CodeXGLUE with Laya, CHOICE repairs 14 of the 60 NOUL errors, and the per-row oracle reaches 0.783 accuracy against 0.550 for the better fixed action, a headroom of 0.233. The same cell shows why headroom is a poor promise: CHOICE breaks 15 correct NOUL answers, so the net count on the training pool is -1 and the net-repair interval spans zero. The other three binary cells have smaller gross opportunities (headroom 0.067–0.100) and net counts between -6 and $+1$. Across the four cells, the training pool never produces a net repair distinguishable from zero.

Held-out pools do not turn the gross opportunity into a stable net gain. The CodeXGLUE/Laya cell gains 20.0 points on `selection_dev` ($[+1.7, +38.3]$), but the same frozen direction gains only 1.7 points on confirm ($[-18.3, +20.0]$). That gap is a selection-pool effect, not a routing effect: dev is the pool on which the direction was chosen. FinDVer with Laya is the exception that tests the narrative. Its confirm net repair is $+13.6$ points $[+2.0, +24.6]$, and the fixed-action accuracy difference is $+0.136$ $[+0.020, +0.246]$. The corresponding macro-F1 difference, however, is $+0.111$ with an interval that crosses zero, and the better fixed action reverses from NOUL on dev to CHOICE on confirm. The cell therefore contains a real accuracy-level repair signal on confirm, but not a macro-F1 gain, and not a direction that a train-selected policy could have known to exploit. CodeXGLUE with Vela moves the other way: the frozen direction is CHOICE-to-NOUL, the train net is -6, and confirm is exactly zero. FinDVer with Vela is negative on confirm (-2.3 points). Complementarity exists in the action set, but it is sparse, metric-dependent and pool-dependent.

The location of the repairs matters as much as their count. Figure 3 (Appendix A) plots the repair rate against the NOUL probability bin. Repairs concentrate in the middle and high bands, around $p \in [0.4, 0.8]$, and are close to zero in the lowest band. This is the opposite of what a confidence cascade assumes: a threshold rule switches when the model is uncertain, which is precisely where CHOICE is least likely to fix anything, because the rows it can repair are rows where NOUL was con-

Table 1: Action complementarity with the train-selected direction frozen across pools ($n = 60$ per cell). *Headroom* is the accuracy gain of a per-row oracle over the better fixed action on `search_train`; *repair/harmful* counts rows where the alternative fixes or breaks the base decision. Net repair is the paired difference in percentage points (alternative minus base) with a 95% group-bootstrap interval on dev and confirm. Headroom is an upper bound, not a deployable gain.

Task / model	Direction	Headroom	Repair/harmful	Net dev (95% CI)	Net confirm (95% CI)
CodeXGLUE / Laya	Noul→Choice	0.233	14/15	+20.0 [+1.7, +38.3]	+1.7 [-18.3, +20.0]
CodeXGLUE / Vela	Switch→Noul	0.067	4/10	-3.3 [-15.0, +8.3]	+0.0 [-15.0, +15.0]
FinDVer / Laya	Noul→Choice	0.083	5/6	-6.8 [-23.3, +6.0]	+13.6 [+2.0, +24.6]
FinDVer / Vela	Switch→Noul	0.100	6/7	+0.0 [-9.1, +11.5]	-2.3 [-10.3, +5.0]

fident and wrong. It also explains why calibration cannot rescue the router: recalibration moves probability mass monotonically, but it does not move repairable rows from the middle of the distribution to the low-confidence tail.

The second question is whether the incremental value of CHOICE is predictable from the first call. Table 2 reports out-of-fold ranking quality and the net utility of the frozen dev-selected policy. Ranking quality varies widely. Logistic regression reaches 0.674 on CodeXGLUE with Laya and 0.919 on FinDVer with Vela; the tree reaches 0.739 on CodeXGLUE with Laya. On FinDVer with Laya both predictors are near chance (0.455 and 0.466), unsurprising given that the cell has only five training repairs. The policy results are weaker than the ranking results. The best dev policy for CodeXGLUE with Laya is logistic regression with a threshold that switches every row: dev net $+20.0$ points, confirm net $+1.7$ points and a cost-adjusted -3.3. The FinDVer/Vela cell, despite the highest OOF AUROC, chooses a policy with dev net $+3.4$ and confirm net -2.3 (zero repairs, one regression). FinDVer with Laya selects a tree that switches one row and gains 2.3 points on confirm, a point estimate built on a single decision. CodeXGLUE with Vela is positive on dev ($+3.3$) and negative on confirm (-1.7). In no cell does a high ranking score translate into a confirm net utility that is both positive and estimated with enough support to matter.

Calibration is the most natural explanation for a failed probability-based router, so it serves as a control. Table 6 and Figure 4 in Appendix A show that raw probabilities are poorly calibrated on confirm: ECE ranges from 0.090 to 0.290 and Brier scores from 0.218 to 0.287. Temperature scaling, fitted on `search_train`, reduces ECE in three of four cells (to 0.045–0.169) and Brier in all four. Routing is unaffected: raw probabilities give confirm net repairs of -5.0, 0.0, $+2.3$ and -2.3 points, temperature-scaled ones give -1.7, $+1.7$, 0.0 and 0.0, as the invariance argument in Section 3 predicts. The repair AUROC makes the point directly: 0.791, 0.444, 0.336 and 0.095 for the four cells; in the last two the probability ranks repairable rows *below* chance. A calibrated probability of the current answer is not a calibrated probability of the alternative action's value.

The third question is whether search converts the available signal into a deployable policy. Table 3 summarises the comparison. GA versus random search is the cleanest test of optimiser value: with five seeds, four task–model cells and three pools, there are 60 paired macro-F1 comparisons, and none has an interval that excludes zero. GA versus fixed switch-only has five significant positive cells, all on the FinDVer/Vela training pool and all with an effect near $+0.12$; on dev and confirm, none of the 40 comparisons is significant. The preregistered deployment comparison—train-selected GA against fixed switch-only on confirm—is null in every seed-level cell. ES versus random search is null in all 30 comparisons; ES versus fixed switching has three significant training cells and one significant confirm cell (CodeXGLUE/Laya, seed 17, $+0.176$), the only confirm-level positive result, and it is not replicated across seeds. NSGA-II deployment is significantly worse than the fixed baseline in five dev cells and one confirm cell, with no positive cell. The search problem has no stable held-out signal for any optimiser to find.

The trajectory diagnostics explain why more search does not help. In 15 of 21 logged GA/ES runs, best-so-far fitness does not improve after generation zero. The neutral-mutation rate is 0.68–0.73, so most mutations produce a candidate with exactly the same training score. The train–dev rank correlation is -0.56 and -0.51 for the two CodeXGLUE cells, meaning that candidates which

Table 2: Incremental-utility prediction. OOF AUROC uses group-aware three-fold cross-validation on `search_train`; the champion and threshold are chosen on `search_dev` and frozen for `confirm`. Net utility is (repairs − regressions)$/n$ in percentage points, with counts in parentheses; the last column subtracts $\lambda = 0.05$ per extra call.

Task / model	LogReg OOF	Tree OOF	Champion	Dev net	Confirm net	Confirm cost-adj.
CodeXGLUE / Laya	0.674	0.739	logreg	+20.0 (23/11)	+1.7 (17/16)	-3.3
CodeXGLUE / Vela	0.716	0.646	logreg	+3.3 (7/5)	-1.7 (9/10)	-6.4
FinDVer / Laya	0.455	0.466	tree	+1.7 (1/0)	+2.3 (1/0)	+1.5
FinDVer / Vela	0.919	0.794	logreg	+3.4 (4/2)	-2.3 (0/1)	-2.6

Table 3: Search comparisons under the pre-registered protocol. Counts are paired group-bootstrap 95% intervals that exclude zero; "positive" means the evolved policy beats the stated baseline. GA and ES rows use macro-F1 on all three pools with five seeds; the NSGA-II row uses accuracy on 12 dev and 12 confirm cells.

Comparison	Pos.	Neg.	Reading
GA vs. random	0	0	no advantage (60)
GA vs. fixed	5	0	train-only (60)
ES vs. random	0	0	no advantage (30)
ES vs. fixed	4	0	one confirm (30)
NSGA-II vs. fixed	0	6	5 dev, 1 confirm

look better on train tend to look worse on dev, and $+0.44$ and $+0.99$ for the FinDVer cells. Raising the budget from 24 to 48 on CodeXGLUE with Vela does not close the gap: the GA evaluates only 22, 17 and 23 unique genomes under a cap of 48, while random search uses all 48. The search is not merely under-tuned; it is under-informed.

The boundary checks ask whether these conclusions are artefacts of one pool or task. Appendix A reports the transfer table and fault map. Transfer is uniformly unhelpful: of 32 source–target by split units, six have a significant paired difference against the target's fixed switch policy, and all six are negative (two diagonal, four off-diagonal). The fault map is more nuanced: of 15 eligible cells, 13 are confirmed on dev and 10 on confirm. The highest confirm severities are on ProcessBench (Vela/mid 0.833, Laya/mid 0.875, Vela/low 0.936, Laya/high 0.895). The mid-confidence cells are not simply the low-confidence tail: a model can be moderately confident and still fail, and the second representation fails on the same rows, so no gate on the first action's probability can create a repair.

Taken together, the three questions describe a routing problem that fails before the optimiser is asked to solve it: RQ1 finds gross but no stable net complementarity, RQ2 finds ranking signal without reliable policy value, and RQ3 finds no held-out advantage because the landscape is flat and the training ranking does not predict the dev ranking where it matters.

The practical implication is a change in the order of work. Before building a router, measure whether the action set has net complementarity on held-out groups and whether the candidate features rank repairable rows above chance. If either check fails, a stronger optimiser will not help; if both pass, the optimiser comparison can be run under a matched budget with a pre-registered confirm comparison.

6 Conclusion

A probability returned with every typed judgment should decide when a second judgment is worth buying. Our experiments on frozen Laya and Vela across three tasks do not support that promise: evolutionary search produces no held-out advantage over fixed policies or budget-matched random search.

Gross complementarity is real—a per-row oracle gains up to 23.3 accuracy points—but harmful switches offset repairs, and the net balance is small, metric-dependent and unstable across pools. Repairs concentrate at mid and high NOUL probability, whereas confidence cascades switch at low probability. Calibration and action-value prediction dissociate: temperature scaling improves ECE and Brier but not row order, and in both FinDVer cells the probability ranks repairable rows below chance. The search landscape is flat: most mutations are neutral, most runs plateau at generation zero, and the training ranking does not predict the dev ranking.

Limits are part of the result: pools are small, the NOUL template was not audited, only two open models were run, and the locked test remains unrun. The next step is a supervision target judged by held-out net utility.

References

Pranjal Aggarwal, Aman Madaan, Yiming Yang, et al. 2024. AutoMix: Automatically mixing language models. In *Advances in Neural Information Processing Systems 37*.

Xavier Bouthillier, Pierre Delaunay, Mirko Bronzi, Assya Trofimov, Jordan Vasiljevic, César Di Bello, et al. 2021. Accounting for variance in machine learning benchmarks. In *Proceedings of Machine Learning and Systems*, volume 3.

Lingjiao Chen, Matei Zaharia, and James Zou. 2024. FrugalGPT: How to use large language models while reducing cost and improving performance. *Transactions on Machine Learning Research*.

C. K. Chow. 1970. On optimum recognition error and reject tradeoff. *IEEE Transactions on Information Theory*, 16(1):41–46.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training verifiers to solve math word problems. *Preprint*, arXiv:2110.14168.

Mehul Damani, Idan Shenfeld, Andi Peng, Andreea Bobu, and Jacob Andreas. 2025. Learning how hard to think: Input-adaptive allocation of LM computation. In *International Conference on Learning Representations*.

Kalyanmoy Deb, Amrit Pratap, Sameer Agarwal, and T. Meyarivan. 2002. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, 6(2):182–197.

Jasper Dekoninck, Maximilian Baader, and Martin Vechev. 2025. A unified approach to routing and cascading for LLMs. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *PMLR*, pages 12987–13010.

Tobias Deußer, Rajkumar Ramamurthy, Lorenz Sparrenberg, and Rafet Sifa. 2026. Evaluating and benchmarking the System One model Jev. *Preprint*, arXiv:2609.37647.

Dujian Ding et al. 2025. BEST-Route: Adaptive LLM routing with test-time optimal compute. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *PMLR*, pages 13870–13884.

Jesse Dodge, Suchin Gururangan, Dallas Card, Roy Schwartz, and Noah A. Smith. 2019. Show your work: Improved reporting of experimental results. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, pages 2185–2194.

Bradley Efron and Robert J. Tibshirani. 1993. *An Introduction to the Bootstrap*. Chapman and Hall/CRC.

Thomas Elsken, Jan Hendrik Metzen, and Frank Hutter. 2019. Neural architecture search: A survey. *Journal of Machine Learning Research*, 20(55):1–21.

Chrisantha Fernando, Dylan S. Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. 2024. Promptbreeder: Self-referential self-improvement via prompt evolution. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *PMLR*.

Zhimin Gao and Pichao Wang. 2026. OpenJev-RLCD: A working RLCD implementation. *Preprint*, arXiv:2609.38850.

Yonatan Geifman and Ran El-Yaniv. 2017. Selective classification for deep neural networks. In *Advances in Neural Information Processing Systems 30*.

Tilmann Gneiting and Adrian E. Raftery. 2007. Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association*, 102(477):359–378.

Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. 2017. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *PMLR*, pages 1321–1330.

Qingyan Guo, Rui Wang, Junliang Guo, Bei Li, Kaitao Song, Xu Tan, Guoqing Liu, Jiang Bian, and Yujiu Yang. 2024. Connecting large language models with evolutionary algorithms yields powerful prompt optimizers. In *International Conference on Learning Representations*.

Ruoqi Guo, Yi Liu, Gelei Deng, Yuekang Li, Lida Zhao, Yutao Wu, Simin Chen, Ying Zhang, and Leo Yu Zhang. 2026. Just ask Jev: Reinforcement learning for calibrated decisions as a zero-shot detector of AI alignment failures. *Preprint*, arXiv:2609.29429.

Nikolaus Hansen and Andreas Ostermeier. 2001. Completely derandomized self-adaptation in evolution strategies. *Evolutionary Computation*, 9(2):159–195.

Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. 2018. Deep reinforcement learning that matters. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32.

Wittawat Jitkrittum, Neha Gupta, Aditya Krishna Menon, Harikrishna Narasimhan, Ankit Singh Rawat, and Sanjiv Kumar. 2023. When does confidence-based cascade deferral suffice? In *Advances in Neural Information Processing Systems 36*.

Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tran-Johnson, et al. 2022. Language models (mostly) know what they know. *Preprint*, arXiv:2207.05221.

Amita Kamath, Robin Jia, and Percy Liang. 2020. Selective question answering under domain shift. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 5684–5696.

Laya Project. 2026. Laya: Non-autoregressive System 1 decision engine, version 0.4.16. https://github.com/NandhaKishorM/laya/tree/b9fe351ce81e2d974bca2bf219c1e4030c0a05ea. Source revision b9fe351; accessed 2026-11-01.

Laya Studio. 2026. Proper scoring rules: How to reward honest probabilities. https://laya.studio/learn/proper-scoring-rules. Accessed 2026-11-01.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2024. Let's verify step by step. In *International Conference on Learning Representations*.

Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin Clement, Dawn Drain, Daxin Jiang, Duyu Tang, et al. 2021. CodeXGLUE: A machine learning benchmark dataset for code understanding and generation. In *NeurIPS 2021 Datasets and Benchmarks Track*.

Quinn McNemar. 1947. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157.

Isaac Ong, Amjad Almahairi, Vincent Wu, et al. 2024. RouteLLM: Learning to route LLMs with preference data. *Preprint*, arXiv:2406.18665.

Esteban Real, Alok Aggarwal, Yanping Huang, and Quoc V. Le. 2019. Regularized evolution for image classifier architecture search. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, pages 4780–4789.

Kenneth O. Stanley and Risto Miikkulainen. 2002. Evolving neural networks through augmenting topologies. *Evolutionary Computation*, 10(2):99–127.

TypeSafe AI. 2026. Introducing System One models and Jev. https://typesafe.ai/blog/introducing-system-one-models-and-jev. Accessed 2026-11-01.

Jonathan Uesato, Nate Kushman, Ramana Kumar, Francis Song, Noah Siegel, Lisa Wang, Antonia Creswell, Geoffrey Irving, and Irina Higgins. 2022. Solving math word problems with process- and outcome-based feedback. *Preprint*, arXiv:2211.14275.

Paul Viola and Michael Jones. 2001. Rapid object detection using a boosted cascade of simple features. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 511–518.

vllm-sr. 2026. Vela 2.0 0.3b: Model card and training details. https://huggingface.co/vllm-sr/Vela-2.0-0.3B. Model revision d6f03aa; accessed 2026-11-01.

Benjamin Warner, Antoine Chaffin, Benjamin Clavié, Orion Weller, Oskar Hallström, Said Taghadouini, et al. 2024. Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference. *Preprint*, arXiv:2412.13663.

Edwin B. Wilson. 1927. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212.

Ji Xin, Raphael Tang, Yaoliang Lee, and Jimmy Lin. 2021. The art of abstention: Selective prediction and error conditioning in question answering. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing*, pages 1040–1051.

Kevin Yang, Dan Klein, Asli Celikyilmaz, Nanyun Peng, and Yuandong Tian. 2023. RLCD: Reinforcement learning from contrast distillation for language model alignment. *Preprint*, arXiv:2307.12950.

Yilun Zhao et al. 2024. FinDVer: Explainable claim verification over long and hybrid-content financial documents. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 14739–14752.

Chujie Zheng et al. 2025. ProcessBench: Identifying process errors in mathematical reasoning. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, pages 1009–1024.

Yaqin Zhou, Shangqing Liu, Jingkai Siow, Xiaoning Du, and Yang Liu. 2019. Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks. In *Advances in Neural Information Processing Systems 32*.

A Appendix

Model and action interfaces. Table 4 summarises the three structured decision models discussed in the paper. Only Laya and Vela are run locally; Jev defines the commercial reference point and is not evaluated here.

Native single-action baselines. Table 5 reports the accuracy and mean per-call latency of the two native actions on the `search_dev` pools. These numbers define the fixed policies that every searched policy is compared against. The two representations have similar per-call cost, which is why the fitness function uses call count rather than a model-specific latency term. Peak resident

Table 4: The three structured decision models discussed in this paper. Only Laya and Vela are run locally; Jev defines the commercial reference point and is not evaluated here.

Model	Interface	Training route	Role in this study
Jev	One call returns typed NOUL, CHOICE and SCORE decisions with probabilities	TypeSafe names RLCD and a proper-scoring objective; the full recipe is not public	Reference point for the RLCD / System One family; not run
Laya 0.4.16	ModernBERT-large encoder with decision heads, ≈421M parameters; NOUL, CHOICE, SCORE	Upstream describes the recipe as RLCD with log, spherical and ranked probability scores	Open RLCD model used in all experiments
Vela 2.0 0.3B	22-layer ModernBERT, 768 hidden, 8,192 tokens; NOUL, CHOICE, SCORE, SPAN, SET	Initialised from Decision-1.0-Kai Choice path, task-specific losses, routing data; not called RLCD	Interface-matched comparison model with a different recipe

memory is included to document the CPU-only execution envelope.

Complementarity overview. Figure 2 visualises the gross and net repair counts behind Table 1, including the train-selected direction applied unchanged to dev and confirm.

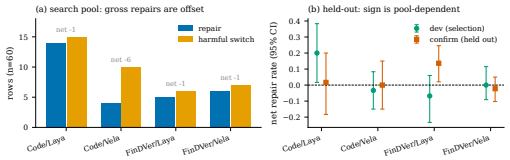


Figure 2: Action complementarity. (a) On the training pool, the second action repairs errors but also breaks correct answers; the net count is printed above each pair. (b) The same train-selected direction applied to dev and confirm: the sign of net repair changes across pools, and only one confirm cell has an interval above zero.

Repair profile. Figure 3 shows the repair rate of the second action as a function of the NOUL probability bin, separately for the training, dev and confirm pools. Each point is the fraction of rows in that bin where NOUL was wrong and CHOICE was correct. Bin counts are printed when at least eight rows fall in the bin. The figure motivates the claim in Section 5 that repairs concentrate at mid and high probability rather than in the low-confidence tail.

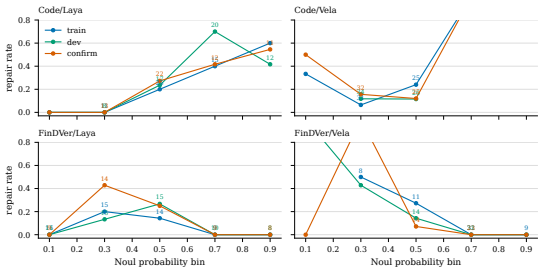


Figure 3: Repair rate by NOUL probability bin for the four binary cells. Repairs are rare in the lowest bin and concentrate around $p \in [0.4, 0.8]$, where a confidence cascade is least likely to switch.

Calibration details. Table 6 reports calibration and its relation to routing; Figure 4 plots reliability diagrams on confirm for the four binary cells, before and after temperature scaling. The scaling parameters are fitted on `search_train` and applied unchanged to confirm. Because temperature scaling is monotone, it changes the numerical probabilities but not the ordering of rows, and therefore not the set of band policies available to the router.

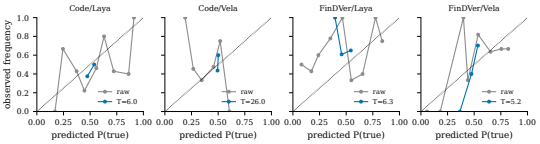


Figure 4: Reliability diagrams on the confirm pool for the four binary cells. Grey is the raw NOUL probability, blue is temperature-scaled. Temperature scaling reduces the calibration error, but it is a monotone map: the ordering of rows, and therefore the set of band policies available up to a reparameterisation, is unchanged.

Search diagnostics. Figure 5 collects the three trajectory diagnostics discussed in Section 5: the neutral mutation rate, the rank correlation between training and dev fitness across the archive, and best-so-far training fitness for seed 17.

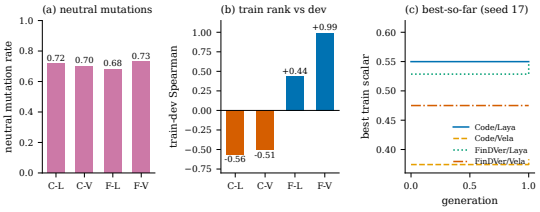


Figure 5: Search diagnostics. (a) Between 68% and 73% of single-gene mutations leave training fitness unchanged. (b) Rank correlation between training and dev fitness across the archive is negative for both CodeXGLUE cells and positive for FinDVer. (c) Best-so-far training fitness for seed 17; most runs improve at most once, and several are flat from generation zero.

Ranking versus policy value. Figure 6 plots out-of-fold AUROC against the confirm net util-

Table 5: Native single-action baselines on the 60-row selection pools (`search_dev`, seed 17). Latency is the per-call mean on the local CPU and is reported only to show that the two representations cost roughly the same; it is not used in the fitness function.

Task	Model	n	NOUL acc.	CHOICE acc.	NOUL ms	CHOICE ms	Peak RSS (MB)
CodeXGLUE	Laya	60	0.450	0.650	2304	2239	2114
CodeXGLUE	Vela	60	0.633	0.667	2437	2341	1931
FinDVer	Laya	59	0.712	0.644	2768	2741	2117
FinDVer	Vela	59	0.458	0.458	2394	2263	1795
ProcessBench	Laya	60	0.100	–	2616	–	2124
ProcessBench	Vela	60	0.100	–	1577	–	1747

Table 6: Calibration on `confirm` and its relation to routing. Temperature T is fitted on `search_train`; ECE uses 15 equal-width bins. Gold AUROC is the probability's ability to rank current-action correctness; repair AUROC is its ability to rank rows that the second action would fix. The last two columns are the net repair of a band policy selected on `search_dev` with raw and temperature-scaled probabilities.

Task / model	ECE raw	ECE T	Brier raw	Brier T	AUROC gold	AUROC repair	Route raw	Route T
CodeXGLUE / Laya	0.230	0.076	0.287	0.250	0.550	0.791	-5.0	-1.7
CodeXGLUE / Vela	0.090	0.045	0.260	0.250	0.522	0.444	+0.0	+1.7
FinDVer / Laya	0.290	0.141	0.285	0.246	0.603	0.336	+2.3	+0.0
FinDVer / Vela	0.147	0.169	0.218	0.234	0.605	0.095	-2.3	+0.0

ity of the frozen dev-selected policy for each cell. Points to the right of 0.5 have ranking signal; points below zero have negative policy value. The dissociation is visible: the cell with the strongest ranking signal (FinDVer/Vela) has a negative confirm net, and the cell with the best confirm net (FinDVer/Laya) has near-chance ranking signal.

Boundary checks. Table 7 reports the transfer counts and the four highest-severity fault-map cells. Figure 7 shows the full confirm-pool severity map by task, model and confidence band.

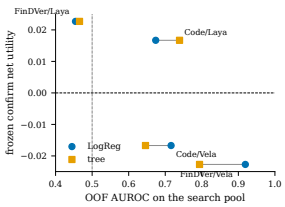


Figure 6: OOF AUROC versus confirm net utility. The dashed horizontal line is zero net utility; the dotted vertical line is chance-level AUROC. Ranking quality and policy value are not the same quantity.

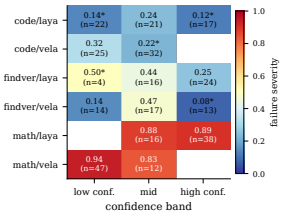


Figure 7: Confirm-pool failure severity by task, model and confidence band. Asterisks mark cells that were not confirmed within the 0.15 tolerance. The stable high-severity region is the mid-confidence band on ProcessBench, where both representations fail together.

Reproducibility details. All results in this paper are derived from committed JSON files. The per-row caches are one JSON file per pool, named `cache_<split>_<task>_<model>.json` under `results/v4/evolution/`; the analyses are produced by `scripts/analyze_action_complementarity.py`, `analyze_complementarity_heldout.py`, `analyze_utility_predictability.py`, `analyze_search_landscape.py`, `analyze_transfer_heldout.py`, `analyze_calibration.py`, `analyze_seed_extension.py`, `analyze_fixed_action_cis.py` and `analyze_search_dynamics.py`. The tables are generated by `scripts/make_paper_tables.py` and the figures by `scripts/plot_paper_figures.py`; neither script contains a hand-copied number. The GA/ES/NSGA-II runners are in `src/evolution/`. Model revisions, dataset revisions, seeds and CPU thread counts are recorded in each result file's metadata. The five-seed extension uses seeds 17, 23, 41, 59 and 73; the budget probe uses the same cache with a cap of 48 unique genomes. No hosted CI runner is used: every model call was executed on the local Windows machine, and the cached outputs are the only inputs to the analyses reported here.

What is not reported. The locked test was not run, the NOUL template was not audited against alternative wordings, real end-to-end latency was not measured for every searched policy, and Jev was not evaluated. These omissions are stated in

Table 7: Boundary checks. Top: frozen-genome transfer, counting significant paired accuracy differences against the target's fixed switch-only policy; diagonal cells are same-task deployment checks, off-diagonal cells are true transfer. Bottom: the four highest-severity fault-map cells, with severity defined as the error rate in the cell.

Transfer	Sig. positive	Sig. negative
Diagonal (same task)	0	2
Off-diagonal (true transfer)	0	4

Fault-map cell	n disc.	Severity	n dev	Severity	n conf.	Severity
ProcessBench / Vela / mid	14	0.929	10	0.800	12	0.833
ProcessBench / Laya / mid	13	0.923	15	0.867	16	0.875
ProcessBench / Vela / low	45	0.889	46	0.957	47	0.936
ProcessBench / Laya / high	41	0.756	35	0.914	38	0.895

the main text and are not filled with proxy numbers. The FinDVer/Laya confirm cell is the only cell with a positive net-repair accuracy interval; it should be treated as a lead for a larger replication, not as a deployable result.